Motivation for Score-Based Generative Modeling

Diffusion ≈ learning s(x, t) and sampling with it.

$$s(\mathbf{x}) = \nabla_{\mathbf{x}} \log p(\mathbf{x})$$

Overview

1. **The Generative Landscape**
   VAE, GAN, and normalizing flows — and the wish list that motivates diffusion.

2. **Energy Models & Normalization**
   Why maximum-likelihood training is intractable: the partition function Z.

3. **Score Matching**
   Learning $\nabla_x \log p$ without Z (Hyvärinen) — and the trace bottleneck.

4. **Scalable Estimators**
   Hutchinson's trick and sliced score matching make it tractable.

5. **Denoising Score Matching**
   Vincent's identity, the DSM objective, and multi-scale noise.

6. **Modern Diffusion & Sampling**
   ε-prediction parameterization and Langevin dynamics.

Appendix — full derivations: MLE gradient, Hyvärinen, Vincent, change of variables, Hutchinson.

PART 01

The Generative Landscape

VAE · GAN · Normalizing flows · The wish list

The Generative Modeling Goal

OBJECTIVE

Learn a model of the data distribution from finite samples to generate new, high-quality data.

$$x \sim p_{\text{data}}(x) \longrightarrow p_\theta(x) \approx p_{\text{data}}(x)$$

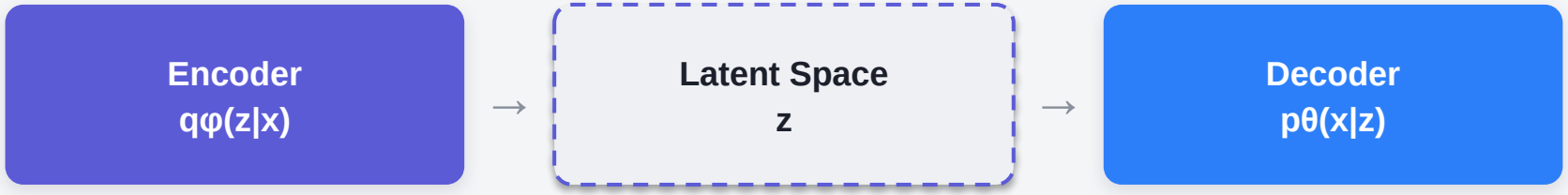
Density Estimation

- Estimate the probability density p(x) for any x.
- Useful for anomaly detection, evaluation, and understanding data structure.
- Requires a normalized or at least tractable objective.

Sample Generation

- Generate new samples x′ that look like they came from pdata(x).
- Requires efficient sampling mechanisms (e.g., GAN generators, Langevin).
- The core of modern creative AI applications.

VAE: What It Optimizes



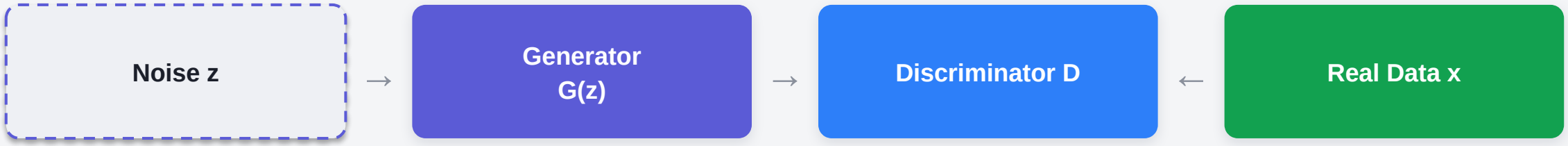
$$\log p_\theta(x) \;\geq\; \mathbb{E}_{q_\phi(z|x)}\big[\log p_\theta(x \mid z)\big] \;-\; \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big)$$

ELBO gap = KL(qφ(z|x) ‖ pθ(z|x))

The VAE Trade-off: Optimizing a lower bound (ELBO) is stable and gives an explicit objective, but samples are often blurry due to the approximate posterior and reconstruction loss.

[1] Kingma & Welling, 2014

GAN: What It Optimizes



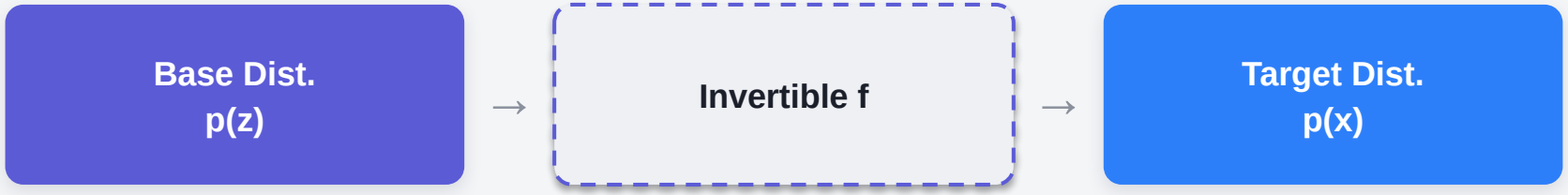
$$\min_G \max_D \; \mathbb{E}_{x \sim p_{\text{data}}}[\log D(x)] \; + \; \mathbb{E}_{z \sim p_z}[\log(1 - D(G(z)))]$$

Equilibrium: pg = pdata and D(x) = 1/2

The GAN Trade-off: GANs generate sharp, high-quality samples, but training is unstable (a minimax game), suffers from mode collapse, and gives no explicit likelihood.

[2] Goodfellow et al., 2014

Normalizing Flows: Exact Likelihood



$$\log p_x(x) = \log p_z\big(f^{-1}(x)\big) + \log \left| \det \nabla_x f^{-1}(x) \right|$$

Change of Variables: the Jacobian determinant accounts for volume distortion under the invertible map.

The Flow Trade-off: Flows enable exact likelihood and efficient sampling, but require invertible architectures with tractable Jacobians — severely limiting expressivity.

See the appendix for the detailed change-of-variables derivation.

[3] Rezende & Mohamed, 2015

The Generative Model Wish List

DESIRED FEATURE	VAE	GAN	FLOW	THE IDEAL MODEL
Sharp Samples	✗ Blurry	✓ Sharp	✓ Sharp	✓ Sharp
Stable Training	✓ Stable	✗ Unstable	✓ Stable	✓ Stable
Explicit Likelihood	~ Bound	✗ No	✓ Exact	✓ Yes
No Arch. Constraints	✓ Any	✓ Any	✗ Invertible	✓ Any
Computational Feasibility	✓ Tractable	✓ Tractable	✗ Costly	✓ Tractable

Diffusion Models aim to check all of these boxes.

PART 02

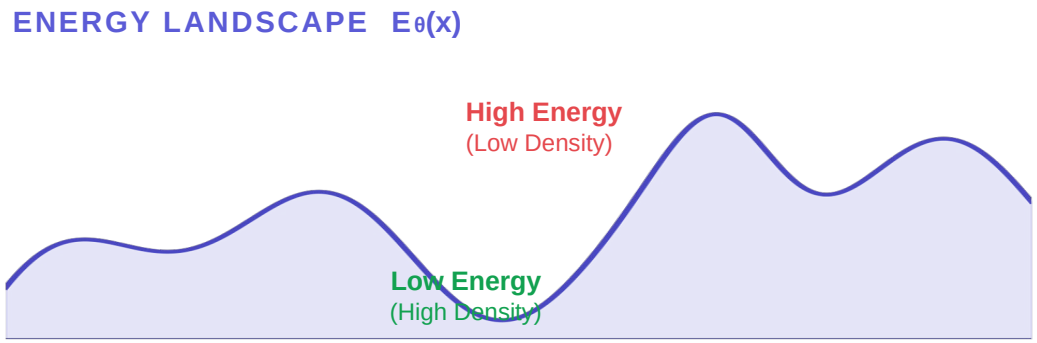
Energy Models & Normalization

The partition function Z · The score function

The Normalization Constant Problem

$$p_\theta(x) = \frac{e^{-E_\theta(x)}}{Z_\theta}$$

$$Z_\theta = \int e^{-E_\theta(x)}\, dx$$



Probability density falls off exponentially as energy rises.

The MLE Gradient Problem

View Derivation ↗

Directly maximizing the log-likelihood log pθ(x) for x ~ pdata(x) is hard because the gradient of the normalization constant Zθ involves an intractable expectation:

$$\nabla_\theta \log Z_\theta = -\mathbb{E}_{x\sim p_\theta}\left[\nabla_\theta E_\theta(x)\right]$$

Score Function: The Constant Cancels

THE SCORE FUNCTION

$$s_\theta(x) = \nabla_x \log p_\theta(x)$$

1. Unnormalized model: $\log p_\theta(x) = -E_\theta(x) - \log Z_\theta$

2. Gradient w.r.t. x: $\nabla_x = \nabla_x(-E_\theta(x)) - \cancel{\nabla_x \log Z_\theta}$

3. Result: $s_\theta(x) = -\nabla_x E_\theta(x)$

Naive Score Matching Loss $\mathcal{L}(\theta) = \mathbb{E}_{x \sim p_{\text{data}}}\left[\|s_\theta(x) - \nabla_x \log p_{\text{data}}(x)\|^2\right]$

The Challenge We don't know ∇_x log pdata(x) — we only have samples, not the density!

PART 03

Score Matching

Hyvärinen’s objective · The trace bottleneck

ICML 2005: Score Matching

“Estimation of Non-Normalized Statistical Models by Score Matching”

Aapo Hyvärinen (2005)

THE TRACTABLE OBJECTIVE

View Full Derivation ↗

$$J_{\mathrm{SM}}(\theta) = \mathbb{E}_{p_{\mathrm{data}}(x)}\left[\frac{1}{2}\|s_\theta(x)\|^2 + \mathrm{tr}\big(\nabla_x s_\theta(x)\big)\right]$$

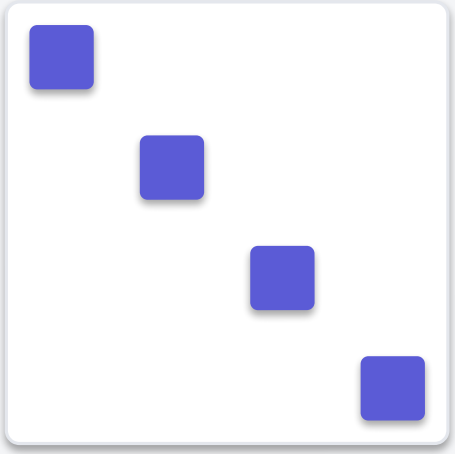
This loss can be computed using only samples from the data distribution and the model’s score function — no normalization constant Zθ or unknown data density pdata is required.

Why Score Matching is Still Hard

THE TRACE BOTTLENECK

The Hyvärinen objective requires computing the divergence (trace of the Jacobian) of the score function:

$$\mathrm{tr}\big(\nabla_x s_\theta(x)\big) = \sum_{i=1}^{d} \frac{\partial s_i}{\partial x_i}$$



O(d) Complexity

For high-dimensional data (e.g., 1024×1024 images), computing this trace is **prohibitively slow.**

We need a way to estimate this trace without computing all d diagonal elements explicitly.

The Trace Bottleneck

$$\nabla_x s_\theta(x) = \nabla_x^2 \log p_\theta(x) = \begin{bmatrix} \partial s_1/\partial x_1 & \partial s_1/\partial x_2 & \partial s_1/\partial x_d \\ \partial s_2/\partial x_1 & \partial s_2/\partial x_2 & \partial s_2/\partial x_d \\ \partial s_d/\partial x_1 & \partial s_d/\partial x_2 & \partial s_d/\partial x_d \end{bmatrix}$$

THE PROBLEM

To compute the trace, we need **d separate backward passes** — one for each input dimension x_i.

THE SCALE

For 1024×1024 images, **$d \approx 10^6$.** Computing the exact trace is impossible on modern GPUs.

PART 04

Scalable Estimators

Hutchinson’s trick · VJP / JVP · Sliced score matching

Hutchinson's Trace Estimator

$$\operatorname{tr}(A) \;=\; \mathbb{E}_{v \sim p(v)}\left[\, v^\top A\, v \,\right]$$

REQUIREMENT FOR v

The random vector v must satisfy: $\mathbb{E}[v] = 0$ and $\mathbb{E}[vv^\top] = I$

Common choices: Rademacher ($v \in \{-1, 1\}^d$) or Gaussian ($v \sim N(0, I)$).

"We replace the sum of d diagonal elements with an expectation over quadratic forms."

View the Proof ↗

[5] Hutchinson, 1990

The VJP / JVP Trick: No Jacobian Needed

$$v^\top \nabla_x s_\theta(x)\, v \;=\; v^\top \nabla_x \big(s_\theta(x)^\top v\big)$$

This is a **Vector–Jacobian Product (VJP)** followed by a dot product.

METHOD	EXACT TRACE	STOCHASTIC ESTIMATOR
Jacobian Matrix	**Requires Full Matrix**	**Never Materialized**
Memory Cost	**$O(d^2)$**	**$O(d)$**
Compute Cost	**d backward passes**	**1 backward pass**

Modern autodiff libraries (JAX, PyTorch) compute VJPs in a single pass.

Sliced Score Matching

$$J_{\mathrm{SSM}}(\theta) = \mathbb{E}_{p_{\mathrm{data}}(x)}\, \mathbb{E}_{p(v)} \left[\frac{1}{2} \left(v^\top s_\theta(x) \right)^2 + v^\top \nabla_x s_\theta(x)\, v \right]$$

where v is a random vector satisfying $\mathbb{E}[vv^\top] = I$

THE “SLICED” IDEA

Instead of matching the full **d-dimensional** score vector, we match the **1D projection** (slice) of the score in random directions v.

EFFICIENT COMPUTATION

Using the Hutchinson trick, this objective needs only **first-order gradients** (VJPs), making it scalable to massive datasets and high-res images.

“We bypass the second-order pain by projecting it into a scalar problem.”

[6] Song, Garg, Shi & Ermon, 2019

SSM Limitations

$$J_{\text{SSM}}(\theta) = \mathbb{E}_{p_{\text{data}}}\, \mathbb{E}_{p(v)} \left[\frac{1}{2} \left(v^\top s_\theta(x) \right)^2 + v^\top \nabla_x s_\theta(x)\, v \right]$$

We've solved the **Trace Problem**, but at a cost.

HIGH VARIANCE

The double expectation over data and v makes training noisy and slow to converge.

STABILITY

Score matching is still unstable in low-density regions where the model has never seen data.

"Can we make the score a simple supervised target?"

PART 05

Denoising Score Matching

Vincent’s identity · The DSM objective · Multi-scale noise

The Direct Learning Signal

Can we make the score a simple supervised target?

$$J_{\text{Ideal}}(\theta) = \mathbb{E}_{p_{\text{data}}}\left[\frac{1}{2}\|s_\theta(x) - s_{\text{data}}(x)\|^2\right]$$

THE BOTTLENECK

We don’t know sdata(x). We only have samples from pdata(x).

THE SOLUTION

We need a way to create a “ground truth” score that we can regress against.

Supervised Denoising

The Denoising Autoencoder (DAE)

$$\mathcal{L}_{\text{DAE}} = \mathbb{E}_{p_{\text{data}}} \mathbb{E}_{p(\tilde{x}|x)} \left[\| f_\theta(\tilde{x}) - x \|^2 \right]$$

Clean Data
$\mathbf{x}$

→

Noisy Input
$\tilde{\mathbf{x}}$

→

Prediction
$\mathbf{f_\theta(\tilde{x})}$

“We know the ground truth x, so we can supervise the reconstruction.”

[7] Vincent, Larochelle, Bengio & Manzagol, 2008

Vincent 2011: Denoiser $\Longleftrightarrow$ Score

$$\nabla_{\tilde{x}} \log p_\sigma(\tilde{x}) = \frac{1}{\sigma^2}\left(\mathbb{E}[x \mid \tilde{x}] - \tilde{x}\right)$$

“The score of the smoothed density is exactly proportional to the optimal denoising direction.”

Want to see the full derivation?

View Full Derivation (Appendix) ↗

[8] Vincent, 2011

Modern DSM Training Loss

The Denoising Score Matching Objective

$$\mathcal{L}_{\mathrm{DSM}}(\theta) = \mathbb{E}_{p_{\mathrm{data}}}\, \mathbb{E}_{p(\tilde{x}|x)} \left[\| s_\theta(\tilde{x}, \sigma) - \nabla_{\tilde{x}} \log p(\tilde{x} \mid x) \|^2 \right]$$

SUPERVISED TARGET

The target $\nabla_{\tilde{x}} \log p(\tilde{x}|x)$ is just the noise direction $(x - \tilde{x})/\sigma^2$.

NO TRACE NEEDED

This is a simple **L2 regression.** No second-order gradients, no Jacobian trace, no pain.

Multi-Sigma: The Denoising Ladder

Multiple Noise Levels

$$\sigma_1 > \sigma_2 > \sigma_3 > \cdots > \sigma_L \approx 0$$

NOISE	COARSE	FINE	DATA
σ_1	σ_2	σ_3	σ_L

“We learn to denoise at every scale, from pure noise to fine data details.”

[11] Song & Ermon, 2019

PART 06

Modern Diffusion & Sampling

ε-prediction · The score field · Langevin dynamics

Modern Diffusion Parameterization

ε-Prediction Objective

$$\mathcal{L}_{\text{simple}}(\theta) = \mathbb{E}_{x,\varepsilon,\sigma}\left[\|\varepsilon - \varepsilon_\theta(x_\sigma, \sigma)\|^2\right]$$

THE EQUIVALENCE

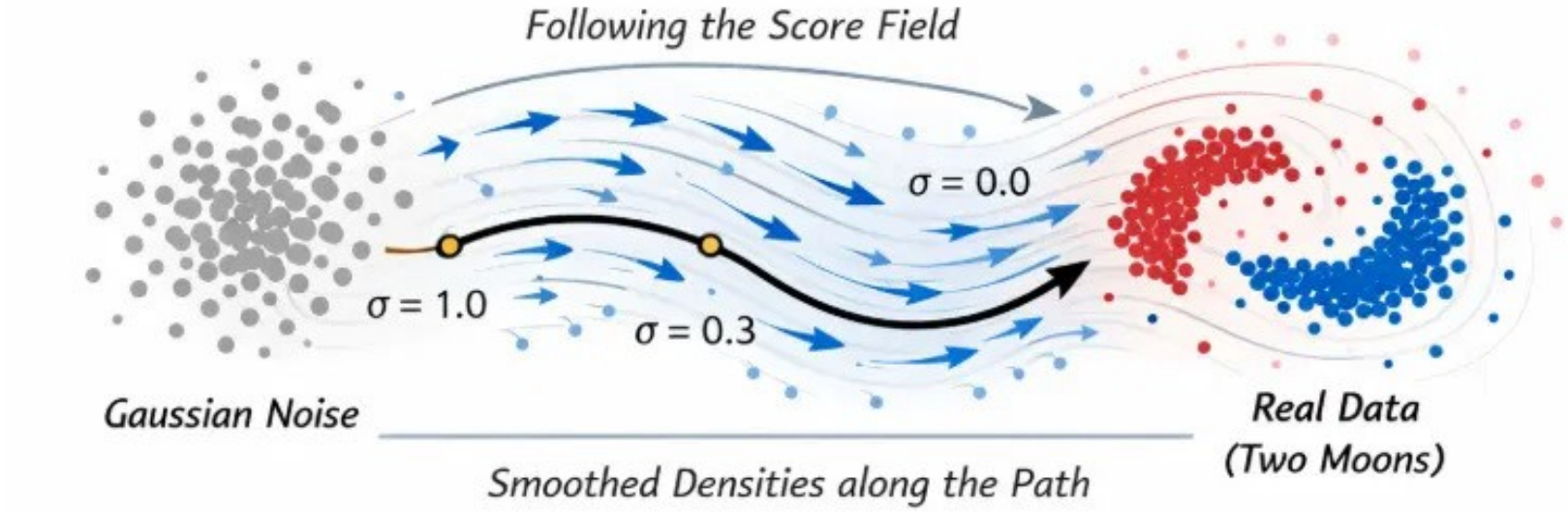
Predicting the noise ε is mathematically equivalent to predicting the score or the clean image x.

NUMERICAL STABILITY

In practice, predicting the unit-variance noise is more stable than predicting a score that can explode as $\sigma \to 0$.

[12] Ho, Jain & Abbeel, 2020

Following the Score Field



“The score field acts as a guide, pushing noisy points from high-entropy regions back toward the Two Moons data distribution.”

Langevin Dynamics: Sampling from the Score

The Langevin Update Rule

$$x_{t+1} = x_t + \eta \nabla_x \log p(x_t, \sigma) + \sqrt{2\eta}\,\varepsilon_t$$

GRADIENT ASCENT

The term $\nabla_x \log p(x_t)$ pushes the sample toward regions of high probability density.

NOISE INJECTION

The term $\sqrt{(2\eta)}\,\varepsilon_t$ prevents the sampler from getting stuck in local modes, ensuring exploration.

[10] Welling & Teh, 2011

Three Equivalent Parameterizations

APPROACH	SCORE PREDICTION	NOISE PREDICTION	DATA PREDICTION
What we model	$s_\theta(\tilde{x}, \sigma)$	$\varepsilon_\theta(\tilde{x}, \sigma)$	$\hat{x}_\theta(\tilde{x}, \sigma)$
Objective	$\|s_\theta - \nabla \log p\|^2$	$\|\varepsilon_\theta - \varepsilon\|^2$	$\|\hat{x}_\theta - x\|^2$
Stability	Can Explode (σ → 0)	Most Stable	Good for Low SNR

“While mathematically equivalent, noise prediction has become the industry standard for stable, high-quality diffusion models.”

APPENDIX

Appendix

Full derivations and supporting proofs

MLE gradient · Hyvärinen score matching · Vincent’s identity · Change of variables · Hutchinson trace

Appendix: MLE Gradient Derivation

GRADIENT OF THE LOG-PARTITION

$$\nabla_\theta \log Z_\theta = \frac{1}{Z_\theta} \nabla_\theta \int e^{-E_\theta(x)} dx = \frac{1}{Z_\theta} \int \big(-\nabla_\theta E_\theta(x)\big) e^{-E_\theta(x)} dx = -\mathbb{E}_{x \sim p_\theta}\big[\nabla_\theta E_\theta(x)\big]$$

$$\nabla_\theta \mathcal{L}(\theta) = -\sum_{i=1}^{n} \nabla_\theta E_\theta(x_i) + n\, \mathbb{E}_{x \sim p_\theta}\big[\nabla_\theta E_\theta(x)\big]$$

Positive Phase (Data): push energy DOWN on real data points to increase their probability.

Negative Phase (Model): push energy UP on model samples to decrease their probability.

← Back to Energy Models

Appendix: Hyvärinen Derivation (Part 1)

STEP 1 — SETUP & GOAL

$$\text{Score:} \quad s_\theta(x) = \nabla_x \log p_\theta(x) = -\nabla_x E_\theta(x)$$

$$\text{Goal:} \quad J(\theta) = \tfrac{1}{2} \int p_{\text{data}}(x) \, \| s_\theta(x) - s_{\text{data}}(x) \|^2 \, dx$$

STEP 2 — EXPAND THE SQUARE

$$J(\theta) = \tfrac{1}{2}\mathbb{E}_{p_{\text{data}}}\left[\|s_\theta(x)\|^2\right] - \mathbb{E}_{p_{\text{data}}}\left[s_\theta(x)^\top s_{\text{data}}(x)\right] + \tfrac{1}{2}\mathbb{E}_{p_{\text{data}}}\left[\|s_{\text{data}}(x)\|^2\right]$$

The third term is constant in θ. The second (highlighted) term is the “nasty” one.

← Back to Score Matching

Next: Part 2 →

Appendix: Hyvärinen Derivation (Part 2)

STEP 3A — INTEGRATION BY PARTS

$$\int s_{\theta,i} \nabla_{x,i} p_{\text{data}} \, dx = \left[s_{\theta,i} p_{\text{data}} \right]_{\partial} - \int p_{\text{data}} \nabla_{x,i} s_{\theta,i} \, dx$$

$$\Rightarrow \int s_{\theta,i} \nabla_{x,i} p_{\text{data}} \, dx = - \int p_{\text{data}} \nabla_{x,i} s_{\theta,i} \, dx$$

The boundary term vanishes.

STEP 3B — SCORE MATCHING

$$\mathbb{E}_{p_{\text{data}}} \left[s_\theta^\top s_{\text{data}} \right] = \int p_{\text{data}} \sum_i s_{\theta,i} \nabla_{x,i} \log p_{\text{data}} \, dx$$

$$= \sum_i \int s_{\theta,i} \nabla_{x,i} p_{\text{data}} \, dx = - \mathbb{E}_{p_{\text{data}}} \left[\nabla_x \cdot s_\theta(x) \right]$$

STEP 4 — FINAL OBJECTIVE

$$J_{\text{SM}}(\theta) = \mathbb{E}_{p_{\text{data}}} \left[\frac{1}{2} \| s_\theta(x) \|^2 + \nabla_x \cdot s_\theta(x) \right]$$

← Part 1

Score Matching →

Appendix: Vincent Derivation (Part 1)

STEP 1 — GAUSSIAN SMOOTHING & SCORE

$$p_\sigma(\tilde{x}) = \int p(x)\,\mathcal{N}(\tilde{x}; x, \sigma^2 I)\,dx \qquad \nabla_{\tilde{x}}\, p_\sigma(\tilde{x}) = \int p(x)\,\nabla_{\tilde{x}}\mathcal{N}(\tilde{x}; x, \sigma^2 I)\,dx$$

STEP 2 — DIFFERENTIATE THE KERNEL

$$\nabla_{\tilde{x}} \log \mathcal{N}(\tilde{x}; x, \sigma^2 I) = -\frac{(\tilde{x} - x)}{\sigma^2} = \frac{(x - \tilde{x})}{\sigma^2}$$

$$\nabla_{\tilde{x}}\, p_\sigma(\tilde{x}) = \int p(x)\,\mathcal{N}(\tilde{x}; x, \sigma^2 I)\left[\frac{(x - \tilde{x})}{\sigma^2}\right] dx$$

← Back to Denoising

Next: Part 2 →

Appendix: Vincent Derivation (Part 2)

STEP 3 — CONVERT TO SCORE

$$\nabla_{\tilde{x}} \log p_\sigma(\tilde{x}) = \frac{\nabla_{\tilde{x}}\, p_\sigma(\tilde{x})}{p_\sigma(\tilde{x})} = \frac{\int p(x)\, \mathcal{N}(\tilde{x}; x, \sigma^2 I)\, (x - \tilde{x})/\sigma^2\, dx}{p_\sigma(\tilde{x})}$$

STEP 4 — IDENTIFY BAYES' POSTERIOR

$$p(x \mid \tilde{x}) = \frac{p(x)\, \mathcal{N}(\tilde{x}; x, \sigma^2 I)}{p_\sigma(\tilde{x})} \quad \Rightarrow \quad \nabla_{\tilde{x}} \log p_\sigma(\tilde{x}) = \int \frac{(x - \tilde{x})}{\sigma^2}\, p(x \mid \tilde{x})\, dx$$

← Back to Part 1

Next: Part 3 →

Appendix: Vincent Derivation (Part 3)

STEP 5 — LINEARITY OF EXPECTATION

$$\nabla_{\tilde{x}} \log p_\sigma(\tilde{x}) = \int \frac{(x - \tilde{x})}{\sigma^2} p(x \mid \tilde{x})\, dx = \frac{1}{\sigma^2}\left[\int x\, p(x \mid \tilde{x})\, dx - \tilde{x} \int p(x \mid \tilde{x})\, dx\right]$$

STEP 6 — THE FINAL IDENTITY

$$\nabla_{\tilde{x}} \log p_\sigma(\tilde{x}) = \frac{1}{\sigma^2}\left[\mathbb{E}[x \mid \tilde{x}] - \tilde{x}\right]$$

Intuition: the score points from the noisy sample $\tilde{x}$ directly toward the posterior mean of the clean data.

← Back to Part 2

Back to Denoising →

Appendix: Change of Variables (Part 1 — Probability Conservation)

The change-of-variables formula is core to normalizing flows: it describes how densities move under an invertible mapping. We start from the requirement that probability is conserved.

SETUP

$$z \sim p_Z(z) \quad \text{(base distribution)}$$

$$x = f(z), \quad \text{with inverse } z = f^{-1}(x)$$

PROBABILITY CONSERVATION

For any measurable set A in x-space:

$$P(x \in A) = P(f(z) \in A) = P(z \in f^{-1}(A))$$

Key idea: the probability of a region is identical whether measured in z-space or x-space — the starting point for the derivation.

Next: rewrite these equalities as integrals and apply the multivariate substitution rule to get the explicit formula.

Appendix: Change of Variables (Part 2 — Derivation)

Step 1 — Express as Integrals

$$\int_A p_X(x)\,dx = \int_{f^{-1}(A)} p_Z(z)\,dz$$

Rewrite conservation using integrals over both domains.

Step 2 — Multivariate Substitution

$$\int_A p_X(x)\,dx = \int_A p_Z(f^{-1}(x))\left|\det \nabla_x f^{-1}(x)\right| dx$$

Volume elements transform by the Jacobian determinant.

Step 3 — Match Integrands

$$p_X(x) = p_Z(f^{-1}(x))\left|\det \nabla_x f^{-1}(x)\right|$$

Holds for all A, so integrands are equal a.e.

Step 4 — Take Logarithms

$$\log p_X(x) = \log p_Z(f^{-1}(x)) + \log\left|\det \nabla_x f^{-1}(x)\right|$$

The change-of-variables formula for exact log-likelihoods.

Result: this is the exact log-likelihood used to train normalizing flows.

Appendix: Change of Variables (Part 3 — Intuition)

1D INTUITION — STRETCHING & COMPRESSING INTERVALS

Take $x = 2z$ (stretch by 2). If $z \in [0,1]$, then $x \in [0,2]$: the interval doubles, so to preserve probability the density must halve:

$$p_X(x) = p_Z(x/2) \cdot \tfrac{1}{2} = p_Z(f^{-1}(x)) \cdot |f'(z)|^{-1}$$

HIGHER DIMENSIONS: THE JACOBIAN DETERMINANT

In d dimensions, a linear map $x = Az$ stretches or compresses volumes by $|\det A|$.

$$p_X(x) = p_Z(f^{-1}(x)) \cdot \left| \det \nabla_x f^{-1}(x) \right|$$

Key Insight

The Jacobian determinant accounts for how the transformation changes volume. Without it, probability would not be conserved under the change of variables.

Appendix: The Hutchinson Identity (Proof)

PROOF STEPS

$$\mathbb{E}\left[v^\top A v\right] = \mathbb{E}\left[\operatorname{tr}(v^\top A v)\right] \qquad \text{(} v^\top A\, v \text{ is a scalar)}$$

$$= \mathbb{E}\left[\operatorname{tr}(A\, v v^\top)\right] \qquad \text{(cyclic property of trace)}$$

$$= \operatorname{tr}\left(A\, \mathbb{E}[v v^\top]\right) \qquad \text{(linearity of expectation)}$$

$$= \operatorname{tr}(A\, I) = \operatorname{tr}(A) \qquad \text{(since } \mathbb{E}[v v^\top] = I \text{)}$$

By taking an expectation over random directions v, we recover the full trace of any matrix A.

← Back to Estimator

References

[1] **Kingma & Welling (2014).** *Auto-Encoding Variational Bayes.* ICLR.

[2] **Goodfellow et al. (2014).** *Generative Adversarial Nets.* NeurIPS.

[3] **Rezende & Mohamed (2015).** *Variational Inference with Normalizing Flows.* ICML.

[4] **Hyvärinen (2005).** *Estimation of Non-Normalized Statistical Models by Score Matching.* JMLR.

[5] **Hutchinson (1990).** *A Stochastic Estimator of the Trace of the Influence Matrix for Laplacian Smoothing Splines.* Commun. Stat. – Simul. Comput..

[6] **Song, Garg, Shi & Ermon (2019).** *Sliced Score Matching: A Scalable Approach to Density and Score Estimation.* UAI.

[7] **Vincent, Larochelle, Bengio & Manzagol (2008).** *Extracting and Composing Robust Features with Denoising Autoencoders.* ICML.

[8] **Vincent (2011).** *A Connection Between Score Matching and Denoising Autoencoders.* Neural Computation.

[9] **Sohl-Dickstein, Weiss, Maheswaranathan & Ganguli (2015).** *Deep Unsupervised Learning using Nonequilibrium Thermodynamics.* ICML.

[10] **Welling & Teh (2011).** *Bayesian Learning via Stochastic Gradient Langevin Dynamics.* ICML.

[11] **Song & Ermon (2019).** *Generative Modeling by Estimating Gradients of the Data Distribution.* NeurIPS.

[12] **Ho, Jain & Abbeel (2020).** *Denoising Diffusion Probabilistic Models.* NeurIPS.

[13] **Song, Sohl-Dickstein, Kingma, Kumar, Ermon & Poole (2021).** *Score-Based Generative Modeling through Stochastic Differential Equations.* ICLR.